Extended Abstract

Motivation We implemented REINFORCE Leave-One-Out (RLOO) for the Countdown task. The task involves finding an expression that evaluates to a given target value, utilizing standard arithmetic operations (+, -, *, /) and parentheses, and each provided number exactly once. For this task, a reward function was provided which gave a reward of 0.1 for a properly annotated solution (marked in `<answer></answer>` tags), and 1.0 if the solution is also correct. Performance in training on this task was not poor, but after many gradient steps, model performance began to decrease (as measured by the reward function). While overfitting certainly contributed, we also hypothesize that the nature of Advantage calculation in RLOO combined with the reward function's structure leads to suboptimal learning goals for examples where the model fails to find a solution in all samples. In particular, a reward of 0.1 does not indicate good performance reasoning capabilities of the model whatsoever, only that response formatting is correct. As such, we were motivated to introduce a modified reward function which rewarded more aspects of the reasoning process behind the policy's response.

Method We started with our Qwen2.5 0.5B model, fine-tuned for 17 epochs using Supervised Fine-Tuning (SFT) on the Countdown task using the WarmStart dataset. We then introduced a modified reward function which rewarded the model for 1. properly marking the answer with `<answer></answer>` tags, 2. correctly using all of the provided numbers in the expression exactly once, 3. correctly using the basic arithmetic operations and parentheses (syntactical correctness), 4. distance to the target value, and 5. exactly reaching the target value. We then used this modified reward function to re-train the fine-tuned model using RLOO, and compared the performance of the two models using the original vanilla reward function, looking at rate of improvement and signs of overfitting and decreases in performance.

Implementation The modified reward function was implemented as follows:

$$r(s,a) = \begin{cases} +0.05 & \text{if } a \text{ contains } \texttt{<answer></answer>} \text{ tags} \\ +0.1 & \text{if } a \text{ contains number provided in } s \text{ exactly once} \\ +0.25 & \text{if } a \text{ is a syntactically correct expression} \\ +0.25 * d, & d = \max(0, 1 - \frac{10 * |\texttt{eval}(a) - \texttt{target}|}{\texttt{target}}) \text{ for } \texttt{target} \text{ in } s \\ +0.35 & \text{if } \texttt{eval}(a) \text{ is equal to the target value from } s \end{cases}$$

The reward function was implemented as a sum of the above components, giving a maximum reward of 1.0 for a perfect solution, just as the vanilla reward function. This preserves the range of the original reward function.

Results After 640 gradient steps, the modified reward function-trained model achieved an average reward of 0.63 on the held out dataset, while the vanilla reward function-trained model achieved a peak average reward of 0.61 after 480 steps. This represents an 3.2% improvement in peak performance between the two training methods. We also found that the modified reward function-trained model was able to generalize better, as it didn't suffer from overfitting during training. This was reflected in the fact that the modified reward function-trained model achieved peak performance at 640 gradient steps, while the vanilla reward function-trained model's peak performance was at 480 gradient steps before suffering model collapse.

Discussion Our results suggest that the apparent performance collapse we had seen with vanilla RLOO stems less from over-fitting of the model weights and more from the reward's sparsity and the leave-one-out advantage calculation. Under RLOO, each update is driven by the difference between two sampled rewards. When the maximum reward of the k samples is only 0.1, the advantage calculation reinforces the model's ability to add `<answer></answer>` tags, but not its ability to solve the problem. By adding partial rewards for components of solving the problem, we were able to improve the performance of the model under RLOO, as the increased density provided more consistent reward signal to the model, and better correlated that reward signal with the model's ability to solve the problem.

Conclusion We explored the use of a multi-objective scalar reward function to improve the density of the reward space for the Countdown task, and used this reward function to train our fine-tuned model using RLOO. We found that the modified reward function led to an 3.2% improvement in peak performance between the two training methods, with the vanilla reward function-trained model achieving a reward of 0.63 and the modified reward function-trained model achieving a reward of 0.61, both scored using the vanilla reward function. We also found that the modified reward function-trained model was able to generalize better, as it didn't suffer from overfitting during training time. This was reflected in the fact that the modified reward function-trained model achieved peak performance at 640 gradient steps, while the vanilla reward function-trained model's peak performance was at 480 gradient steps.

Multi-Objective Reinforcement Learning & k-Sampling Data Augmentation with RLOO

Coleman Smith
Department of Computer Science
Stanford University
colemansmith@stanford.edu

Abstract

Large language models still falter on structured arithmetic challenges such as Countdown, where a solver must combine a fixed set of numbers exactly once to reach a target value. We show that REINFORCE Leave-One-Out (RLOO) underperforms when its reward function is sparse, nearly binary, and offers almost no advantage signal when all sampled attempts are wrong, but some still achieve a small reward. We introduce a multi-objective reward function for the Countdown task that supplements the original annotation and correctness rewards with rewards for single use of every number, syntactic validity, and distance-to-target. Training from a Fine-Tuned Qwen2.5 0.5B model with this denser signal improves peak evaluation reward by 3.2% over a baseline trained with the vanilla reward and postpones overfitting and model collapse. Our results show that reward function density and shaping can have a significant impact on training performance and success, and poorly shaped reward functions can result in misdirected reinforcement.

1 Introduction

The Countdown task involves generating an arithmetic expression given some set of numbers which, when evaluated, is equal to the given target number. Basic arithmetic (+, -, *, /) and parentheses may be used, and each number must be used exactly once.

We seek to train the Qwen2.5 0.5B model on this task, aiming to improve math reasoning ability. The Countdown task generally involves understanding differences and ratios between the given numbers, and arranging them into an expression with that in mind. Thus, any solution involves a multi-step reasoning process of determining potential solutions and evaluating them for correctness, before presenting a final, ideally correct solution.

We trained the model using two methods. First, we finetuned using Supervised Fine-Tuning with the WarmStart dataset Asap7772 (2025). The aim of this step was to introduce the task to the model, and demonstrate how it should format its reasoning and answering steps. We expect the model to provide its answer enclosed in `<answer></answer>` tags for extraction, so that we can check the answer for correctness using a deterministic reward function provided by GitHub (2025).

We then train the model using RL. Our method for this is the REINFORCE Leave-One-Out algorithm, which involves calculating reward-based advantages for a set of samples on a given prompt, and then using these advantages to calculate an advantage-weighted average of the losses of each sample using the negative log likelihood loss estimator.

2 Related Work

The dominant approach to aligning large language models (LLMs) is reinforcement learning from human feedback (RLHF) Ouyang et al. (2022), often implemented with Proximal Policy Optimization (PPO). Follow-up work has explored variants that reduce the reliance on explicit human preference data, including constitutional AI (RLAIF) Bai et al. (2022), exploratory preference optimisation Xie et al. (2024), and direct REINFORCE-style optimisation with analytically motivated baselines Ahmadian et al. (2024). Our study adopts REINFORCE Leave-One-Out (RLOO) Ahmadian et al. (2024) because it matches PPO's performance while avoiding the use of an auxiliary value network.

Sparse or misaligned rewards are a major bottleneck when applying RL to reasoning problems. Several works augment the scalar reward with automatically checkable sub-objectives: DeepSeek-R1 shapes rewards to promote intermediate verification steps in mathematical derivations DeepSeek-AI et al. (2025), and the cognitive-behaviours framework GitHub (2025) provides rule-based scores (format, verification) for the Countdown task. We build on this idea by adding further automatically-verifiable objectives (syntax validity, full number usage, distance to target) to create a denser reward landscape.

Optimising multiple, sometimes competing criteria has a long history in reinforcement learning; recent work argues that pluralistic alignment of LLMs (accuracy, helpfulness, safety, etc.) requires explicitly multi-objective treatments Sorensen et al. (2024). Our reward can be viewed as a weighted scalarisation of four such objectives, providing an empirical case study of the benefits of multi-objective shaping within the LLM domain.

We follow the CS224R default setup by starting from the Qwen2.5 0.5B base model Qwen Team (2024). Supervised fine-tuning (SFT) uses the WarmStart Countdown solutions dataset Asap7772 (2025), after which RL is conducted on the larger Countdown Prompts dataset Pan (2025). Our work is therefore complementary to prior research that scales RL training on similar resources but focuses on alternate optimisation objectives or sampling strategies.

3 Method

We started with a fine-tuned Qwen 2.5 0.5B model, which we trained on the WarmStart dataset using SFT. We picked the best performing checkpoint, which was after 17 epochs, as our starting point for RL training using RLOO.

We then trained the model using RLOO with the vanilla reward function from GitHub (2025), which provides rewards between 0 and 1. After, we then trained the baseline SFT trained model using RLOO again, but using our modified reward function which provides rewards from 0 to 1, but is denser.

For the modified reward function, we implemented the following reward function:

$$r(s,a) = \begin{cases} +0.05 & \text{if } a \text{ contains } \texttt{<answer></answer>} \text{ tags} \\ +0.1 & \text{if } a \text{ contains number provided in } s \text{ exactly once} \\ +0.25 & \text{if } a \text{ is a syntactically correct expression} \\ +0.25 * d, & d = \max(0, 1 - \frac{10 * |\texttt{eval}(a) - \texttt{target}|}{\texttt{target}}) \text{ for } \texttt{target} \text{ in } s \\ +0.35 & \text{if } \texttt{eval}(a) \text{ is equal to the target value from } s \end{cases}$$

The reward function was implemented as a sum of the above components, giving a maximum reward of 1.0 for a perfect solution, just as the vanilla reward function. This preserves the range of the original reward function.

We trained with RLOO using small batch sizes due to both compute constraints as well as with the intention to promote overfitting in order to see the performance differences between RLOO with the vanilla and modified reward functions, as our hypothesis is that the modified reward function will better maintain model generality and prevent model collapse.

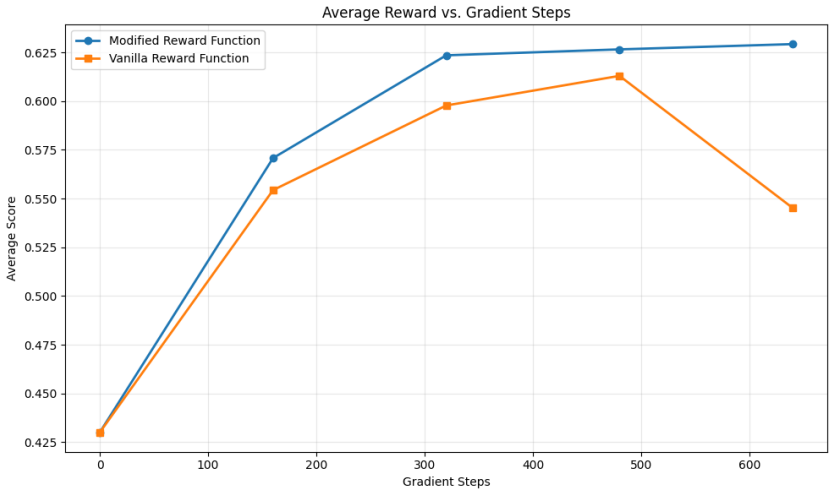

Figure 1: Average Reward of RLOO-trained models vs. Gradient Steps.

4 Experimental Setup

We finetuned the Qwen 2.5 0.5B model on the WarmStart dataset for 20 epochs using Supervised Fine-Tuning (SFT). This dataset provides 1000 fully-worked Countdown solutions in prompt-response pairs to show the policy desirable reasoning patterns and output formatting. Each prompt was formatted in the same way, differing only in the numbers and target provided. We trained with a batch size of 4 examples, and used a learning rate of $1e-5$ with PyTorch's AdamW optimizer. We then evaluated the model after each epoch, with the checkpoint after epoch 17 performing the best. We used this as our baseline for our RLOO training and experiments.

For training with RLOO, we utilized the Countdown Prompts dataset, composed of 490,000 numbers-target examples. We formatted each example using the same format as the prompts in the WarmStart dataset. We trained for 640 gradient steps, using a batch size of 1, learning rate of $1e-5$, and PyTorch's AdamW optimizer. We used a temperature of 0.8 for sampling, and if all samples received the same reward, we resampled using a temperature of 0.9. If again the new samples all received the same score, then the example was skipped, as all the advantages and thus loss would be zero. For each example, we used $k = 16$ examples for calculating the leave-one-out advantage and the example loss. We used the reward function available from GitHub (2025), which provided a reward of 0.1 for a properly annotated solution (marked in `<answer></answer>` tags), and 1.0 if the annotated solution is correct (within 10^{-5} of the target value).

We then trained the model using RLOO with the modified reward function for 640 gradient steps, using the same batch size, learning rate, optimizer, and $k = 16$ samples per example as the baseline RLOO training. However, for this training run, we used our modified reward function outlined in our Methods, which provided a denser reward space over the same range of 0 to 1.

5 Results

We evaluated the performance of the vanilla and modified reward function-RLOO trained models using the held out dataset provided by the CS224R teaching team. We checkpointed the model after every 160 gradient steps, and then evaluated each checkpointed model 10 times over the dataset to achieve an average reward for each checkpoint, including pre-RL training performance. Note that this is not a best-of-k sampling with $k = 10$, but 10 independent trails over the entire held out dataset, where we first take the average reward of each trial, then average these values over the 10 trials. The rewards for these evaluations were calculated using the vanilla reward function to the sake of consistency when comparing the two models. These results are visualized as Average Reward vs. Gradient Steps in Figure 1.

Table 1: Performance Comparison

Method	Initial Average Reward	Peak Average Reward	Final Average Reward
SFT Baseline	0.43	—	—
RLOO Vanilla Reward	0.43	0.61	0.54
RLOO Modified Reward	0.43	0.63	0.63

5.1 Quantitative Evaluation

The initial, peak and final average reward are listed in Table 1.

We found that our baseline model scored an average of 0.43 over 10 trials, our vanilla RLOO training achieved a peak average score of 0.61, and our modified reward function RLOO training achieved a peak average score of 0.63. This represents a 47% improvement over baseline for modified RLOO, and a 3.2% improvement over vanilla RLOO. This demonstrates a marginal improvement in general ability of the model when trained using RLOO with our modified reward function.

Importantly, we can see that the final average reward for the model trained with RLOO using the vanilla reward function was lower than its peak average reward, indicating that performance was beginning to collapse at some point during the training run.

5.2 Qualitative Analysis

Looking at Figure 1, we can see that RLOO with the modified reward function achieved a higher average reward more quickly than with our vanilla reward function. Given that the former provides a denser reward space, this would appear to be explained by the more varied rewards given to the different samples from each example, which may have helped the model learn the task more incrementally compared with the vanilla reward function.

Additionally, we can see that the model trained with the vanilla reward function appears to suffer performance collapse between 480 and 640 gradient steps, with average reward taking a steep decline, losing nearly half the performance gains since the start of training. It is possible that this is partially due to overfitting, but when compared to the average reward of the model trained with RLOO using our modified reward function, which does not display such overfitting or collapse, it seems more likely that this is due to inauspicious advantage weights, which result in the model "unlearning" how to solve the Countdown task, as it's rewarded only for correctly annotating its answer in those cases.

6 Discussion

We were motivated to investigate this line of experimentation by our observations that the rewards being assigned to our samples during RLOO training were often all less than 1. For the vanilla reward function, in cases where all rewards are less than one the advantage weights positively weight samples which properly annotate the solution and negatively weight samples which do not. This means that the model learns nothing about how to solve the underlying reasoning task, Countdown, but only that it must provide `<answer></answer>` tags somewhere in its response. In fact, the response `<answer></answer>` would receive a reward of 0.1 under the vanilla reward function. As such, we can see that 1. the reward space of the vanilla reward function is quite sparse, with samples that are rewarded 0.1 not necessarily providing much for the model to learn from compared to samples that receive a reward of 0, and 2. If the model is quite good at providing answer tags, but bad at reasoning and solving the Countdown task, then it will be reinforced that the model should continue providing answer tags, but rarely if ever be pushed to actually provide a correct answer to the prompt.

Comparatively, the modified reward function gives positive advantage weights for 1. following the rules of the Countdown task, and 2. getting close to the correct answer. Thus, even if the model struggles in reaching the correct answer, it should still get closer to being able to do so as it moves toward achieving higher and higher reward, preventing the inflection point and collapse seen with the vanilla reward function. Effectively, the modifications to the reward function prevent us from reinforcing a necessary but not primary behavior that we want, and help to reinforce almost correct

solutions, allowing the policy to improve incrementally and without getting caught in the trap of prioritizing solutions with reward 0.1 when there are none that solve the prompt perfectly.

Importantly, in cases where all the k samples for an example fail to follow the rules of Countdown and only receive a reward for providing answer tags or otherwise a reward of 0, then we fall into the same issue as with the vanilla reward function. However, given the already relatively good performance of the fine-tuned model, we believe that this is unlikely. Furthermore, it is far easier for the model to learn to list every number provided in the prompt within the answer tags than it is to sole the entire Countdown task, so it's also more likely that if performance does transiently decrease from such a situation, it would be able to recover due to the dense and incremental nature of the reward function.

7 Conclusion

We implemented SFT and RLOO for the purpose of training Qwen2.5 0.5B to perform the Countdown task. We used Supervised Fine-Tuning to introduce the model to the prompt format and problem task, and then used RLOO to promote generalization and better reasoning ability. Using a small batch size to promote overfitting and poor advantage weights, we found that RLOO with our vanilla, sparse reward function leads to model collapse after 640 gradient steps in our experiments. Motivated by our observations of the advantage weights for several examples, we reformulated a reward function which exposed a denser reward space and provided incremental reward as the model came closer to performing and solving the Countdown task correctly. We saw that this modified reward function resulted in slightly better average performance of the trained model, and more importantly that it prevented model collapse in the minimal batch size setting, which is generally more susceptible to overfitting and model collapse.

8 Team Contributions

- **Coleman Smith:** Sole group member, responsible for all parts of the project.

Changes from Proposal Originally, the project was going to explore Synthetic Data Augmentation, Multi-Objective Optimization, and Exploration Strategies as ways of promoting diversity and generalization, particularly on the Instruction Following task. Due to changes in the project scope, the focus of the project shifted to the Countdown task. As such, the project was modified to focus on Multi-Objective Optimization in the Countdown task, which involved modifying the vanilla reward function of RLOO to introduce a denser reward space. There was also some desire to explore k-Sampling Data Augmentation, but due to time/workforce constraints, this was not explored.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Sara Hooker, Olivier Pietquin, Ahmet Üstün, and Julia Kreutzer. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. `https://arxiv.org/abs/2402.14740`. arXiv:2402.14740 [cs.LG]

Asap7772. 2025. WarmStart Countdown Solutions Dataset. `https://huggingface.co/datasets/Asap7772/cog_behav_all_strategies`.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073 [cs.CL] `https://arxiv.org/abs/2212.08073`

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Gou,

Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/abs/2501.12948. arXiv:2501.12948 [cs.CL]

cognitive-behaviors GitHub. 2025. Countdown Rule-Based Reward Implementation. https://github.com/kanishkg/cognitive-behaviors/blob/main/verl/utils/reward_score/countdown.py.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL]

Jiayi Pan. 2025. Countdown Prompts Dataset. https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4.

Qwen Team. 2024. Qwen2.5 Technical Report. https://arxiv.org/abs/2412.15115. arXiv:2412.15115 [cs.CL]

Taylor Sorensen, Jared Moore, Jillian Fisher, Mitchell Gordon, Niloofar Mireshghallah, Christopher Michael Rytting, Andre Ye, Liwei Jiang, Ximing Lu, Nouha Dziri, Tim Althoff, and Yejin Choi. 2024. A Roadmap to Pluralistic Alignment. arXiv:2402.05070 [cs.AI] https://arxiv.org/abs/2402.05070

Tengyang Xie, Dylan J. Foster, Akshay Krishnamurthy, Corby Rosset, Ahmed Awadallah, and Alexander Rakhlin. 2024. Exploratory Preference Optimization: Harnessing Implicit Q*-Approximation for Sample-Efficient RLHF. arXiv:2405.21046 [cs.LG] https://arxiv.org/abs/2405.21046